

Gestion des Incidents et des Crises

Cybersécurité - Mastère 2

De la Détection à la Résolution

Prérequis

Avant de suivre ce cours, vous devez :

- Connaître les **notions fondamentales de la cybersécurité**
- Avoir des **connaissances en gestion de projet**
- Être familier avec la **gestion des risques informatiques**
- Avoir complété les capsules e-learning :
 - "Security Incident Reporting" (4h)
 - "Incident Handling Process" (3h)

Objectifs Pédagogiques

À l'issue de cette formation, vous serez capable de :

- **Comprendre** les étapes clés de la gestion des incidents de cybersécurité
- **Élaborer** et mettre en œuvre des plans d'intervention efficaces
- **Coordonner** les réponses en situation de crise
- **Conduire** des enquêtes post-incident (root cause analysis)
- **Développer** des compétences en prise de décision sous pression
- **Appliquer** des méthodologies et outils adaptés

Programme de la Formation

MODULE 1 - Fondamentaux

- Définitions et taxonomie
- Cycle de vie d'un incident
- Frameworks de référence (NIST, SANS)

MODULE 2 - Détection & Analyse

- Sources de détection
- Triage et classification
- Analyse forensique initiale

MODULE 3 - Réponse aux Incidents

- Confinement et éradication
- Récupération et restauration
- Communication de crise

MODULE 4 - Gestion de Crise

- Activation de la cellule de crise
- Coordination des équipes
- Prise de décision sous pression

MODULE 5 - Post-Incident

- Analyse post-mortem
- Lessons learned
- Amélioration continue

MODULE 6 - Exercices Pratiques

- Simulations d'incidents
- Tabletop exercises
- Outils SOC/CSIRT

Module 1

Fondamentaux de la Gestion des Incidents

Qu'est-ce qu'un Incident de Sécurité ?

Définition NIST SP 800-61

“ Un **incident de sécurité informatique** est une violation ou une menace imminente de violation des politiques de sécurité, des politiques d'utilisation acceptable, ou des pratiques de sécurité standard. ”

Qu'est-ce qu'un Incident de Sécurité ?

Exemples d'incidents

- Accès non autorisé à des systèmes ou données
- Déni de service (DoS/DDoS)
- Infection par malware (ransomware, trojan, etc.)
- Exfiltration de données
- Compromission de comptes utilisateurs

Incident vs Événement vs Alerte



Incident vs Événement vs Alerte

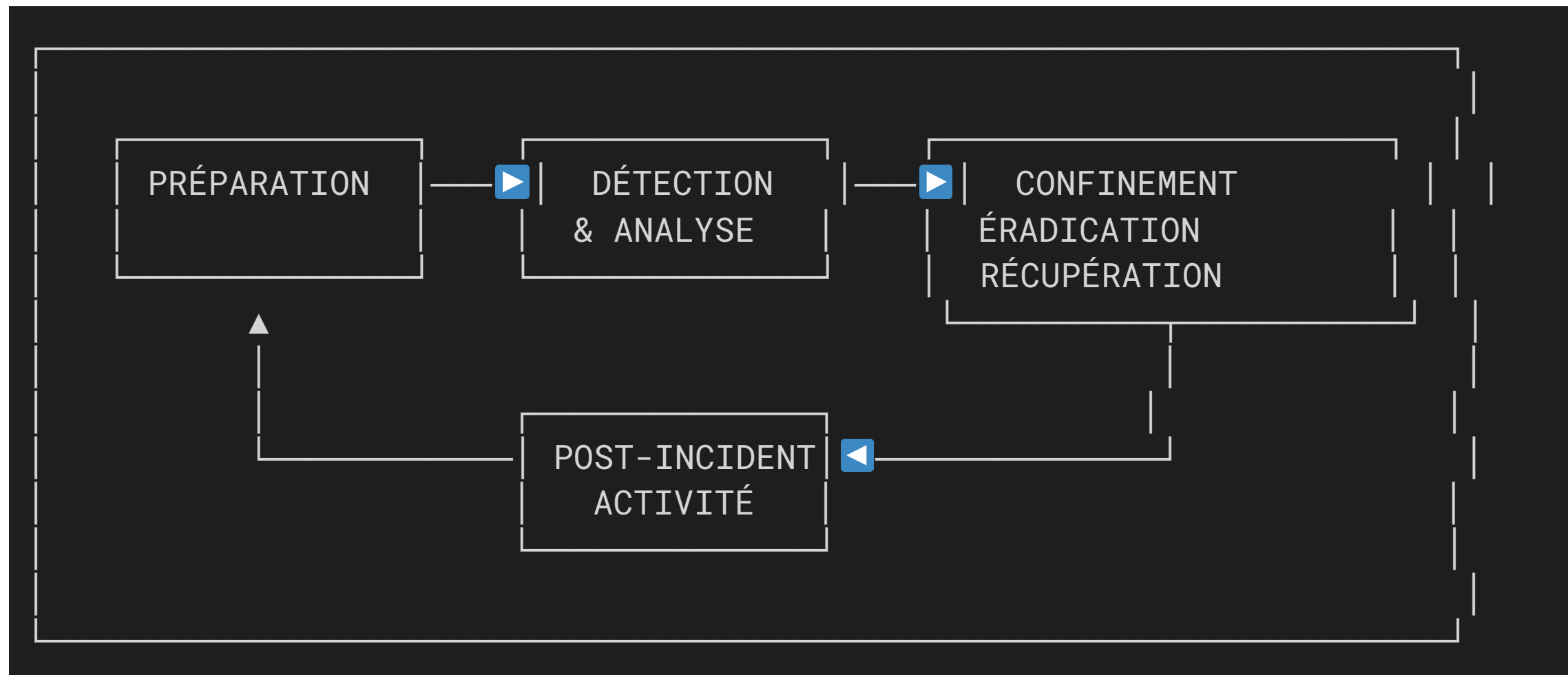
Terme	Définition
Événement	Toute occurrence observable dans un système
Alerte	Événement signalé comme potentiellement malveillant
Incident	Alerte confirmée nécessitant une réponse
Crise	Incident majeur menaçant l'organisation

Taxonomie des Incidents

Classification ENISA

Catégorie	Description	Exemples
Contenu abusif	Spam, harcèlement	Phishing, diffamation
Code malveillant	Malware, virus	Ransomware, trojans, worms
Collecte d'information	Reconnaissance	Scanning, sniffing
Tentative d'intrusion	Exploitation	Brute force, 0-day
Intrusion	Compromission avérée	Backdoor, rootkit
Disponibilité	Déni de service	DDoS, sabotage
Intégrité	Modification données	Défacement, manipulation
Fraude	Usurpation	Phishing, vol d'identité

Le Cycle de Vie d'un Incident (NIST)



Phase 1 : Préparation

Objectif

Établir la capacité à répondre efficacement aux incidents **AVANT** qu'ils ne surviennent.

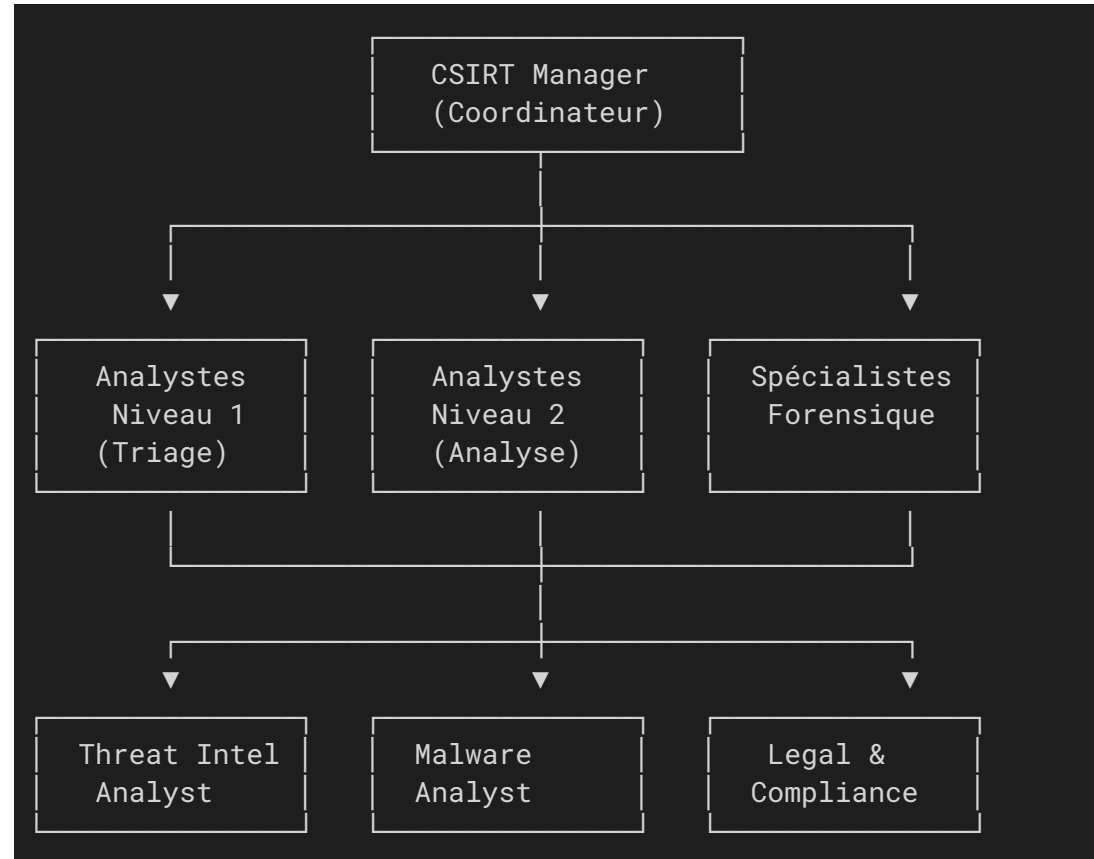
Éléments clés

- Politique de gestion des incidents
- Équipe CSIRT/SOC constituée
- Outils et technologies déployés
- Procédures documentées
- Contacts et escalades définis

Livrables

- Plan de réponse aux incidents (IRP)
- Playbooks par type d'incident
- Matrice de communication
- Kit de réponse (jump bag)
- Environnement de forensique

Composition d'une Équipe CSIRT



Exercice 1 : Créer un Plan de Réponse

Consigne

En groupe, créez la structure d'un **Plan de Réponse aux Incidents (IRP)** pour une entreprise de e-commerce.

Sections à inclure

1. **Objectifs** et périmètre
2. **Rôles et responsabilités** (RACI)
3. **Classification** des incidents
4. **Procédures** de détection et notification
5. **Procédures** de réponse par type
6. **Communication** interne et externe
7. **Indicateurs** de performance (KPI)

Framework SANS - 6 Étapes

Étape	SANS	NIST Équivalent
1	Preparation	Preparation
2	Identification	Detection & Analysis
3	Containment	Containment
4	Eradication	Eradication
5	Recovery	Recovery
6	Lessons Learned	Post-Incident Activity

“ Le framework SANS est plus **opérationnel**, le NIST plus **stratégique** ”

Niveaux de Sévérité

Matrice de Classification

Niveau	Nom	Critères	SLA Réponse	Exemples
P1	Critique	Impact business majeur, données sensibles	< 15 min	Ransomware actif, breach données clients
P2	Élevé	Impact significatif, propagation possible	< 1h	Malware détecté, compte admin compromis
P3	Moyen	Impact limité, système non critique	< 4h	Phishing réussi, scan détecté
P4	Bas	Impact minimal, informatif	< 24h	Tentative de connexion échouée

Démo : Script de Classification Automatique

```
#!/usr/bin/env python3
"""
Script de classification automatique des incidents
Basé sur les critères NIST et matrices de risque
"""

from enum import Enum
from dataclasses import dataclass
from typing import List

class Severity(Enum):
    CRITICAL = 1 # P1
    HIGH = 2 # P2
    MEDIUM = 3 # P3
    LOW = 4 # P4

class ImpactType(Enum):
    CONFIDENTIALITY = "confidentiality"
    INTEGRITY = "integrity"
    AVAILABILITY = "availability"

@dataclass
class Incident:
    title: str
```

Démo : Classification (suite)

```
class IncidentClassifier:
    """Classifie automatiquement la sévérité d'un incident"""

    def classify(self, incident: Incident) -> Severity:
        score = 0

        # Critères de scoring
        if incident.sensitive_data:
            score += 40
        if incident.business_critical:
            score += 30
        if incident.propagation_risk:
            score += 20
        if incident.user_count > 100:
            score += 10

        # Mapping score -> sévérité
        if score >= 70:
            return Severity.CRITICAL
        elif score >= 50:
            return Severity.HIGH
        elif score >= 30:
            return Severity.MEDIUM
        else:
            return Severity.LOW

# Utilisation
incident = Incident(
    title="Ransomware détecté sur serveur de fichiers",
    affected_systems=["FILE-SRV-01", "FILE-SRV-02"],
    data_involved=True,
    sensitive_data=True,
    user_count=500,
    business_critical=True,
    propagation_risk=True
)
```

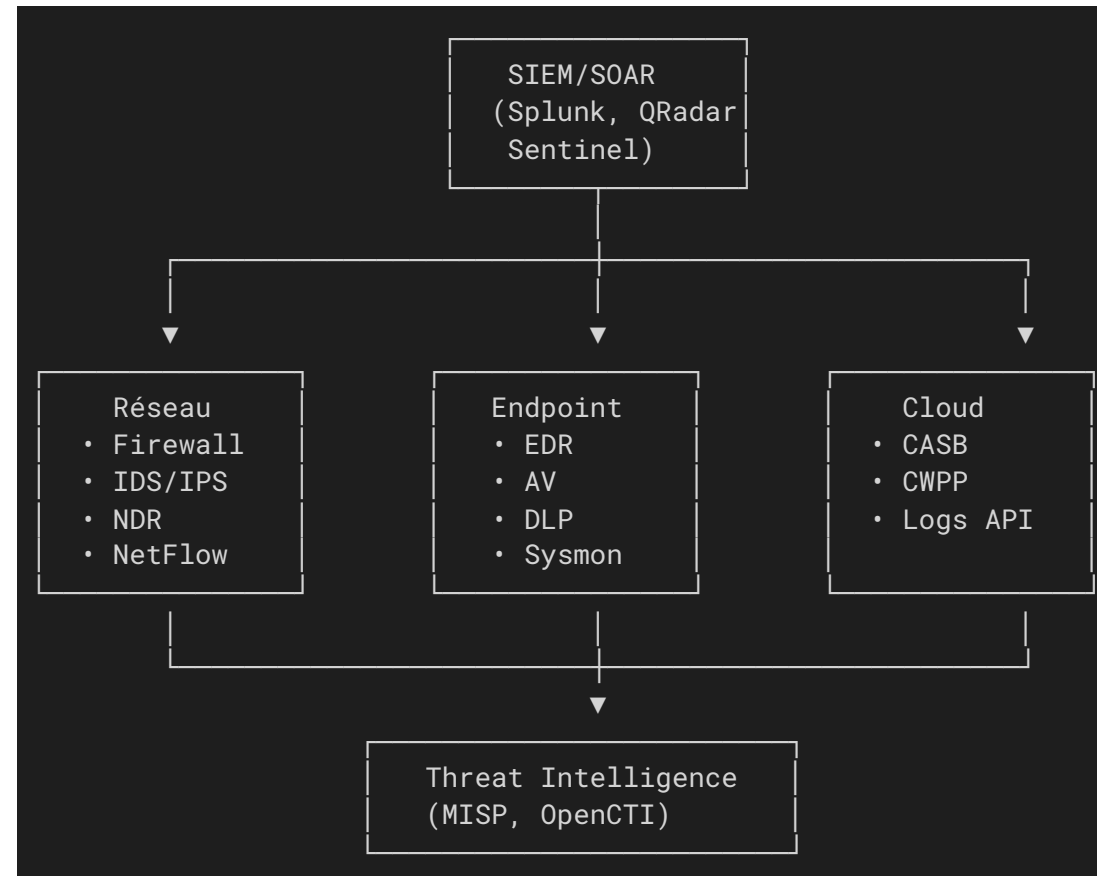
Module 2

Détection et Analyse des Incidents

Sources de Détection

SOURCES DE DÉTECTION		
TECHNIQUES	HUMAINES	EXTERNES
• SIEM • IDS/IPS • EDR/XDR • Antivirus • Logs systèmes • Firewall • WAF • DLP	• Utilisateurs • Helpdesk • Administrateurs • Équipe sécurité • Audit interne	• CERT nationaux • Partenaires • Fournisseurs • Médias • Threat Intelligence • Bug bounty • Forces de l'ordre

Architecture SOC Typique



Indicateurs de Compromission (IoC)

Types d'IoC

Niveau Réseau

- Adresses IP malveillantes
- Domaines C2
- URLs de phishing
- Hash de fichiers
- Patterns de trafic

Niveau Host

- Hash de malware (MD5, SHA256)
- Noms de fichiers suspects
- Clés de registre
- Mutex
- Processus anormaux

Indicateurs de Compromission (IoC)

Formats Standards

- **STIX/TAXII** : Structured Threat Information eXpression
- **OpenIOC** : Format Mandiant
- **YARA** : Règles de détection de malware

Démo : Règle YARA pour Détection

```
rule Ransomware_Generic_Indicators
{
  meta:
    description = "Détection des indicateurs génériques de ransomware"
    author = "SOC Team"
    date = "2024-01-15"
    severity = "high"

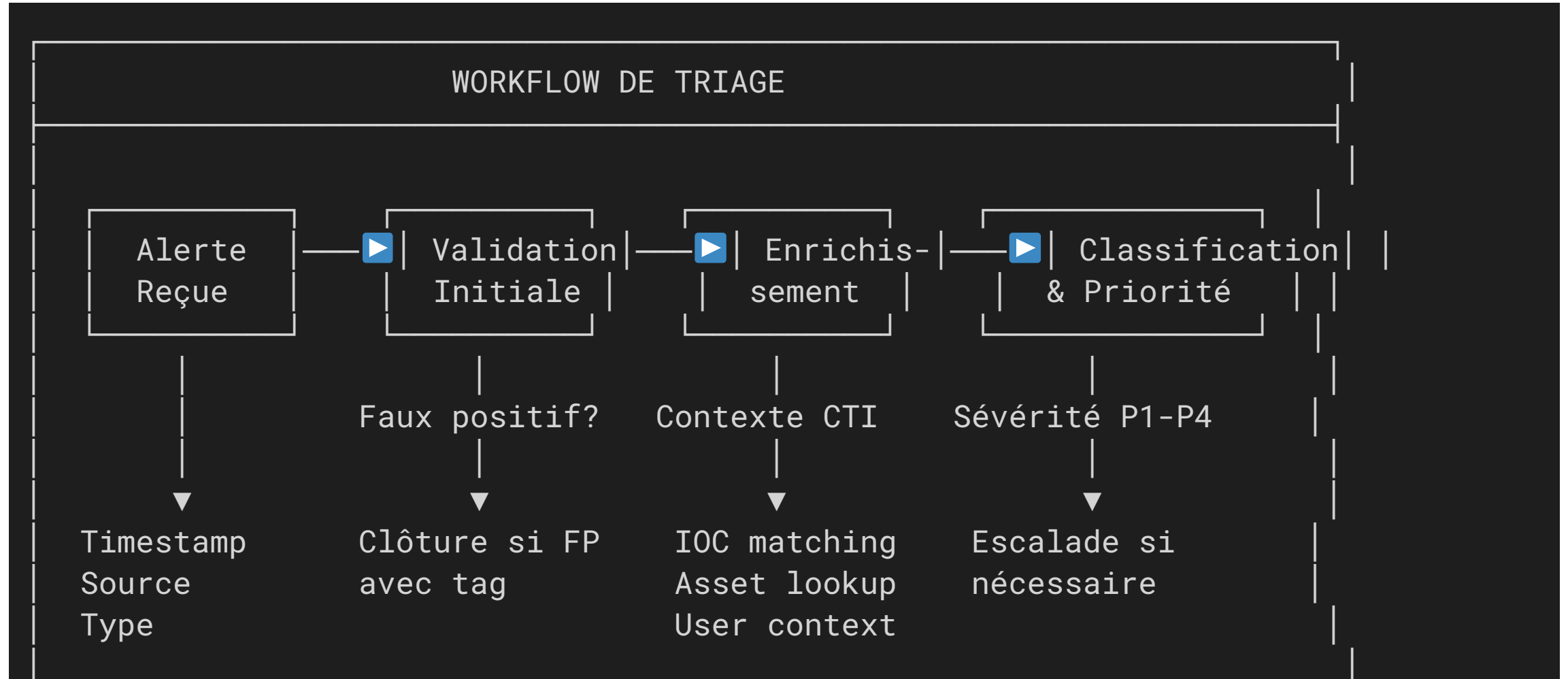
  strings:
    // Extensions de fichiers chiffrés courantes
    $ext1 = ".encrypted" nocase
    $ext2 = ".locked" nocase
    $ext3 = ".crypto" nocase

    // Messages de rançon typiques
    $ransom1 = "Your files have been encrypted" nocase
    $ransom2 = "bitcoin" nocase
    $ransom3 = "decrypt" nocase
    $ransom4 = "pay" nocase

    // APIs Windows suspectes
    $api1 = "CryptEncrypt"
    $api2 = "CryptGenKey"
    $api3 = "GetLogicalDrives"

  condition:
    uint16(0) == 0x5A4D and // MZ header (PE file)
    (2 of ($ext*)) and
    (2 of ($ransom*)) and
    (2 of ($api*))
}
```

Processus de Triage



Démo : Script de Triage Automatisé

```
#!/usr/bin/env python3
"""
Script de triage automatisé des alertes SIEM
Intègre enrichissement CTI et classification
"""

import requests
from datetime import datetime
from typing import Optional, Dict

class AlertTriager:
    def __init__(self, vt_api_key: str, misp_url: str):
        self.vt_api_key = vt_api_key
        self.misp_url = misp_url

    def triage_alert(self, alert: Dict) -> Dict:
        """Processus de triage complet d'une alerte"""

        result = {
            "alert_id": alert["id"],
            "timestamp": datetime.now().isoformat(),
            "status": "pending",
            "enrichments": {}
        }

        # 1. Extraction des IOC
        iocs = self._extract_iocs(alert)

        # 2. Enrichissement VirusTotal
        for ioc in iocs:
            vt_result = self._check_virustotal(ioc)
            result["enrichments"][ioc] = vt_result

        # 3. Vérification MISP
        misp_hits = self._check_misp(iocs)
        result["misp_correlation"] = misp_hits

        # 4. Classification automatique
        result["severity"] = self._classify(result)
```

Démo : Enrichissement CTI

```
def _check_virustotal(self, ioc: str) -> Dict:
    """Vérifie un IOC sur VirusTotal"""

    headers = {"x-apikey": self.vt_api_key}

    # Déterminer le type d'IOC
    if self._is_ip(ioc):
        url = f"https://www.virustotal.com/api/v3/ip_addresses/{ioc}"
    elif self._is_hash(ioc):
        url = f"https://www.virustotal.com/api/v3/files/{ioc}"
    elif self._is_domain(ioc):
        url = f"https://www.virustotal.com/api/v3/domains/{ioc}"
    else:
        return {"error": "Unknown IOC type"}

    try:
        response = requests.get(url, headers=headers, timeout=10)
        if response.status_code == 200:
            data = response.json()
            stats = data.get("data", {}).get("attributes", {})
            return {
                "malicious": stats.get("last_analysis_stats", {}).get("malicious", 0),
                "suspicious": stats.get("last_analysis_stats", {}).get("suspicious", 0),
                "reputation": stats.get("reputation", "unknown")
            }
    except Exception as e:
        return {"error": str(e)}
```

Analyse Forensique Initiale

Collecte de Preuves - Ordre de Volatilité

Priorité	Source	Volatilité	Outil
1	Mémoire RAM	Très haute	Volatility, AVML
2	Cache réseau	Haute	Wireshark, tcpdump
3	Processus actifs	Haute	pslist, handles
4	Sessions réseau	Haute	netstat, ss
5	Fichiers temporaires	Moyenne	Timeline
6	Disque dur	Basse	dd, FTK Imager
7	Logs distants	Basse	SIEM export
8	Sauvegardes	Très basse	Backup system

“ **Règle d'or** : Toujours collecter du plus volatile au moins volatile ”

Démo : Acquisition Mémoire avec AVML

```
#!/bin/bash
# Script d'acquisition mémoire Linux avec AVML
# À exécuter sur le système compromis

# Variables
CASE_ID="INC-2024-001"
ANALYST="analyst@company.com"
EVIDENCE_PATH="/mnt/evidence/${CASE_ID}"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# Création du répertoire de preuves
mkdir -p "${EVIDENCE_PATH}"

# Acquisition mémoire avec AVML (Microsoft)
echo "[*] Démarrage acquisition mémoire..."
./avml "${EVIDENCE_PATH}/memory_${TIMESTAMP}.lime"

# Calcul du hash pour intégrité
echo "[*] Calcul hash SHA256..."
sha256sum "${EVIDENCE_PATH}/memory_${TIMESTAMP}.lime" > \
    "${EVIDENCE_PATH}/memory_${TIMESTAMP}.sha256"

# Métadonnées
cat << EOF > "${EVIDENCE_PATH}/acquisition_metadata.json"
{
    "case_id": "${CASE_ID}",
    "analyst": "${ANALYST}",
    "timestamp": "${TIMESTAMP}",
    "hostname": "$(hostname)",
    "kernel": "$(uname -r)",
    "tool": "AVML",
    "tool_version": "$(./avml --version)"
}
EOF
```

Démo : Analyse Mémoire avec Volatility 3

```
#!/bin/bash
# Analyse mémoire avec Volatility 3

MEMORY_DUMP="./memory_20240115_143022.lime"
OUTPUT_DIR="./analysis_results"

mkdir -p "${OUTPUT_DIR}"

# 1. Informations système
echo "[*] Extraction info système..."
vol -f ${MEMORY_DUMP} linux.pslist > ${OUTPUT_DIR}/pslist.txt

# 2. Connexions réseau actives
echo "[*] Extraction connexions réseau..."
vol -f ${MEMORY_DUMP} linux.sockstat > ${OUTPUT_DIR}/sockstat.txt

# 3. Fichiers ouverts par processus
echo "[*] Extraction fichiers ouverts..."
vol -f ${MEMORY_DUMP} linux.lsof > ${OUTPUT_DIR}/lsof.txt

# 4. Historique bash
echo "[*] Extraction historique bash..."
vol -f ${MEMORY_DUMP} linux.bash > ${OUTPUT_DIR}/bash_history.txt

# 5. Modules kernel chargés
echo "[*] Extraction modules kernel..."
vol -f ${MEMORY_DUMP} linux.lsmod > ${OUTPUT_DIR}/lsmod.txt

# 6. Recherche de malware connu
echo "[*] Scan malware avec YARA..."
vol -f ${MEMORY_DUMP} yarascan.YaraScan \
  yara_file /rules/malware.yar > ${OUTPUT_DIR}/yara_hits.txt
```

Exercice 2 : Analyse d'Alertes SIEM

Scénario

Vous recevez les alertes suivantes de votre SIEM :

```
09:15:23 - ALERT - Multiple failed SSH logins from 185.234.72.xx
09:17:45 - ALERT - Successful SSH login from 185.234.72.xx (user: admin)
09:18:02 - ALERT - Privilege escalation detected (user: admin -> root)
09:18:34 - ALERT - Suspicious outbound connection to pastebin.com
09:19:12 - ALERT - New scheduled task created (cron)
09:22:45 - ALERT - Large data transfer to external IP (2.3 GB)
```

Exercice 2 : Analyse d'Alertes SIEM

Questions

1. Quelle est la **kill chain** de cette attaque ?
2. À quel moment auriez-vous dû **escalader** ?
3. Quelles **actions immédiates** recommandez-vous ?
4. Quels **IOC** extraire pour le blocage ?

MITRE ATT&CK Framework

Utilisation pour l'Analyse

MITRE ATT&CK MATRIX					
Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Exfiltration
T1190 Exploit Public Facing App	T1059 Command & Script Interp	T1053 Scheduled Task	T1548 Abuse Elevation Control	T1070 Indicator Removal	T1041 Exfiltration Over C2 Channel

Démo : Mapping ATT&CK Automatique

```
#!/usr/bin/env python3
"""
Mapping automatique des alertes vers MITRE ATT&CK
"""

from typing import List, Dict

# Base de connaissances simplifiée ATT&CK
ATTACK_MAPPING = {
    "ssh_brute_force": {
        "technique_id": "T1110.001",
        "technique_name": "Brute Force: Password Guessing",
        "tactic": "Credential Access"
    },
    "privilege_escalation_sudo": {
        "technique_id": "T1548.003",
        "technique_name": "Abuse Elevation Control: Sudo",
        "tactic": "Privilege Escalation"
    },
    "scheduled_task_creation": {
        "technique_id": "T1053.003",
        "technique_name": "Scheduled Task/Job: Cron",
        "tactic": "Persistence"
    },
    "data_exfiltration_http": {
        "technique_id": "T1048.003",
        "technique_name": "Exfiltration Over Alternative Protocol: HTTP",
        "tactic": "Exfiltration"
    }
}

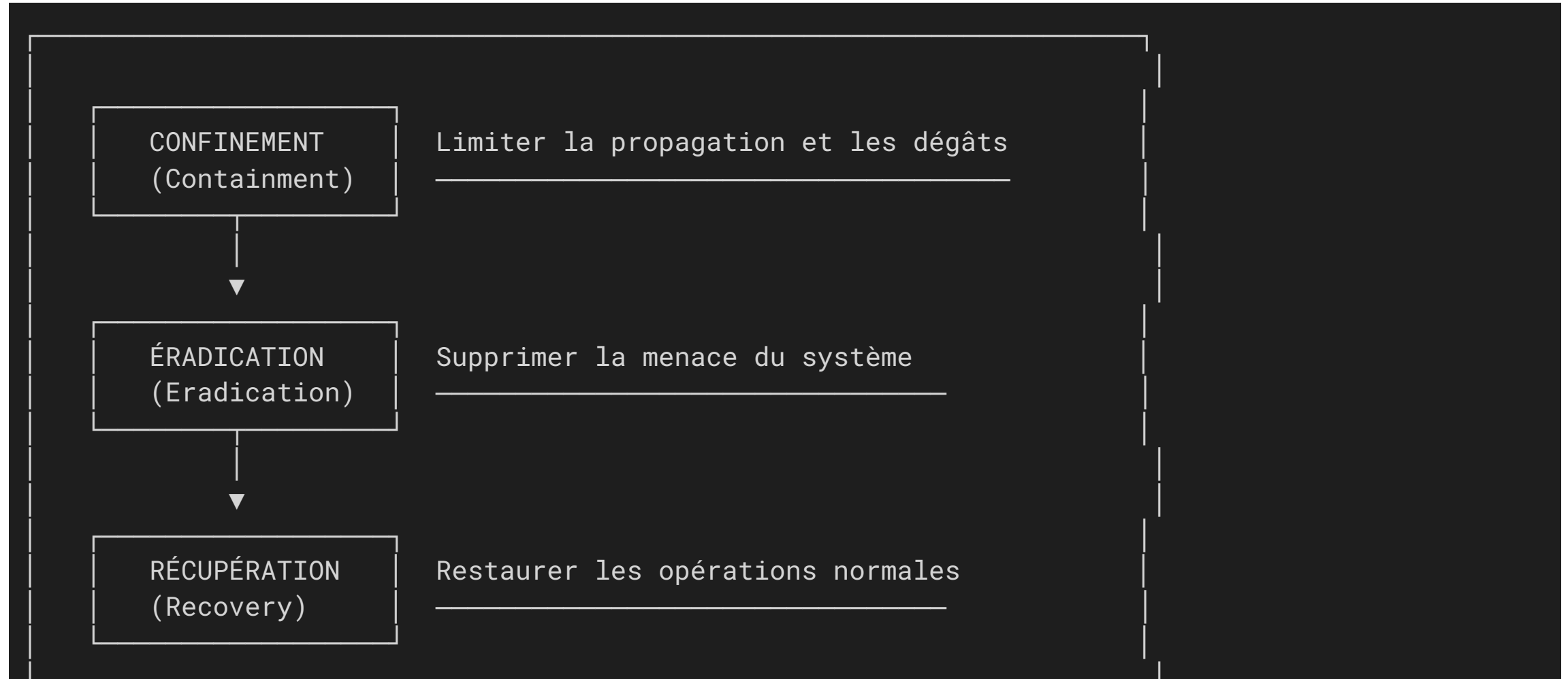
class ATTACKMapper:
    def map_alert(self, alert_type: str) -> Dict:
        """Retourne le mapping ATT&CK pour un type d'alerte"""
        return ATTACK_MAPPING.get(alert_type, {
            "technique_id": "Unknown",
            "technique_name": "Requires manual analysis",
            "tactic": "Unknown"
        })

    def generate_attack_chain(self, alerts: List[str]) -> List[Dict]:
        """Génère la chaîne d'attaque complète"""
        chain = []
        for alert in alerts:
```

Module 3

Réponse aux Incidents

Les 3 Phases de Réponse



Stratégies de Confinement

Court terme vs Long terme

Court terme (immédiat)

- Isolation réseau du système
- Blocage IP/domaine sur firewall
- Désactivation compte compromis
- Arrêt service vulnérable

Long terme (durable)

- Segmentation réseau
- Patch du système vulnérable
- Rotation des credentials
- Mise à jour des règles IDS
- Renforcement des contrôles d'accès

Attention : Ne pas éteindre !

Démo : Script de Confinement Automatisé

```
#!/usr/bin/env python3
"""
Script de confinement automatisé pour incident Linux
À exécuter avec privilèges root sur le système compromis
"""

import subprocess
import logging
from datetime import datetime

class IncidentContainment:
    def __init__(self, incident_id: str):
        self.incident_id = incident_id
        self.log = self._setup_logging()

    def _setup_logging(self):
        logging.basicConfig(
            filename=f'/var/log/incident_{self.incident_id}.log',
            level=logging.INFO,
            format='%(asctime)s - %(levelname)s - %(message)s'
        )
        return logging.getLogger(__name__)

    def isolate_network(self, preserve_forensic: bool = True):
        """Isole le système du réseau tout en préservant les connexions forensiques"""

        self.log.info("Démarrage isolation réseau...")

        # Sauvegarder les règles iptables actuelles
        subprocess.run(['iptables-save', '-f', '/tmp/iptables_backup.rules'])

        if preserve_forensic:
            # Autoriser uniquement le serveur forensique
            forensic_server = "10.0.100.50"
            rules = [
                f"iptables -A INPUT -s {forensic_server} -j ACCEPT",
                f"iptables -A OUTPUT -d {forensic_server} -j ACCEPT",
                "iptables -A INPUT -j DROP",
                "iptables -A OUTPUT -j DROP"
            ]
        else:
            rules = [
                "iptables -A INPUT -j DROP",
                "iptables -A OUTPUT -j DROP"
            ]
```

Démo : Confinement (suite)

```
def disable_compromised_account(self, username: str):
    """Désactive un compte utilisateur compromis"""

    self.log.info(f"Désactivation compte: {username}")

    # Verrouiller le compte
    subprocess.run(['usermod', '-L', username])

    # Expirer le mot de passe
    subprocess.run(['chage', '-E', '0', username])

    # Tuer toutes les sessions de l'utilisateur
    subprocess.run(['pkill', '-u', username])

    # Invalider les tokens SSH
    ssh_dir = f"/home/{username}/.ssh"
    subprocess.run(['rm', '-rf', ssh_dir])

    self.log.info(f"Compte {username} désactivé et sessions terminées")

def block_ioc(self, ioc_type: str, value: str):
    """Bloque un IOC sur le système"""

    if ioc_type == "ip":
        subprocess.run(['iptables', '-A', 'INPUT', '-s', value, '-j', 'DROP'])
        subprocess.run(['iptables', '-A', 'OUTPUT', '-d', value, '-j', 'DROP'])
        self.log.info(f"IP bloquée: {value}")

    elif ioc_type == "domain":
        # Ajouter au hosts file pour blocage DNS
        with open('/etc/hosts', 'a') as f:
            f.write(f"\n127.0.0.1 {value} # Blocked - Incident {self.incident_id}\n")
        self.log.info(f"Domaine bloqué: {value}")

# Utilisation
containment = IncidentContainment("INC-2024-001")
```

Éradication

Checklist d'Éradication

Systèmes compromis

- ☐ Identifier tous les systèmes affectés
- ☐ Supprimer le malware/backdoor
- ☐ Nettoyer la persistance (cron, services, registre)
- ☐ Vérifier l'intégrité des fichiers système
- ☐ Supprimer les comptes créés par l'attaquant

Credentials

- ☐ Réinitialiser les mots de passe compromis
- ☐ Révoquer les tokens/sessions
- ☐ Régénérer les clés API
- ☐ Rotation des certificats si nécessaire
- ☐ Vérifier les accès privilégiés

Important : S'assurer que l'attaquant n'a pas de persistance avant de restaurer les accès !

Démo : Script de Hunting de Persistance

```
#!/bin/bash
# Script de détection de mécanismes de persistance sur Linux
# À exécuter après confinement pour identifier la persistance

REPORT_FILE="/tmp/persistence_hunt_$(date +%Y%m%d_%H%M%S).txt"

echo "=== PERSISTENCE HUNTING REPORT ===" > ${REPORT_FILE}
echo "Date: $(date)" >> ${REPORT_FILE}
echo "" >> ${REPORT_FILE}

# 1. Vérification des tâches cron
echo "=== CRON JOBS ===" >> ${REPORT_FILE}
for user in $(cut -f1 -d: /etc/passwd); do
    crontab -l -u $user 2>/dev/null >> ${REPORT_FILE}
done
cat /etc/crontab >> ${REPORT_FILE}
ls -la /etc/cron.* >> ${REPORT_FILE}

# 2. Services systemd suspects
echo "" >> ${REPORT_FILE}
echo "=== SYSTEMD SERVICES (recently modified) ===" >> ${REPORT_FILE}
find /etc/systemd/system -mtime -7 -type f >> ${REPORT_FILE}

# 3. Fichiers rc.local et init
echo "" >> ${REPORT_FILE}
echo "=== RC.LOCAL ===" >> ${REPORT_FILE}
cat /etc/rc.local 2>/dev/null >> ${REPORT_FILE}

# 4. Authorized keys SSH
echo "" >> ${REPORT_FILE}
echo "=== SSH AUTHORIZED KEYS ===" >> ${REPORT_FILE}
find /home -name "authorized_keys" -exec cat {} \; >> ${REPORT_FILE}
cat /root/.ssh/authorized_keys 2>/dev/null >> ${REPORT_FILE}

# 5. Binaires SUID suspects
echo "" >> ${REPORT_FILE}
echo "=== SUID BINARIES ===" >> ${REPORT_FILE}
```

Récupération et Restauration

Stratégies de Recovery

Stratégie	Description	Quand l'utiliser
Patch & Continue	Corriger et continuer	Compromission mineure, système nettoyable
Rebuild	Reconstruction complète	Compromission profonde, rootkit
Restore from Backup	Restauration sauvegarde	Données corrompues, ransomware
Replace	Remplacement matériel	Compromission firmware/BIOS

Validation avant remise en production

1. Scan antimalware complet
2. Vérification intégrité avec baseline
3. Tests de pénétration ciblés

Communication de Crise

Matrice de Communication

MATRICE DE COMMUNICATION		
DESTINATAIRE	QUAND	CONTENU
Direction	P1: Immédiat P2: < 1h	Impact business, décisions requises, ressources
Juridique	Si données personnelles	Obligations légales, notification CNIL
Communication	Si risque médiatique	Messages pré-approuvés, Q&A préparées
Clients	Si impact service/données	Notification transparente, mesures prises

Template de Notification d'Incident

```
# NOTIFICATION D'INCIDENT DE SÉCURITÉ
```

```
**Classification**: CONFIDENTIEL
```

```
**Incident ID**: INC-2024-XXX
```

```
**Date de notification**: [DATE]
```

```
**Statut**: En cours / Résolu
```

```
## Résumé Exécutif
```

```
[Description en 2-3 phrases de l'incident et son impact]
```

```
## Chronologie
```

```
| Heure | Événement |  
|-----|-----|  
| HH:MM | Détection initiale |  
| HH:MM | Confinement activé |  
| HH:MM | ... |
```

```
## Impact
```

- **Systèmes affectés**: [Liste]
- **Données concernées**: [Type et volume]
- **Utilisateurs impactés**: [Nombre]
- **Impact business**: [Description]

```
## Actions Prises
```

1. [Action 1]
2. [Action 2]

```
## Prochaines Étapes
```

- [] [Action planifiée]

Exercice 3 : Simulation de Réponse

Scénario

09:00 - Un ransomware est détecté sur 3 postes du service comptabilité.

Les fichiers sont chiffrés avec l'extension `.locked`.

Une note de rançon demande 5 BTC.

Travail en groupe

1. **Constituez** votre cellule de crise (rôles)
2. **Définissez** les actions de confinement immédiates

Module 4

Gestion de Crise

Qu'est-ce qu'une Crise Cyber ?

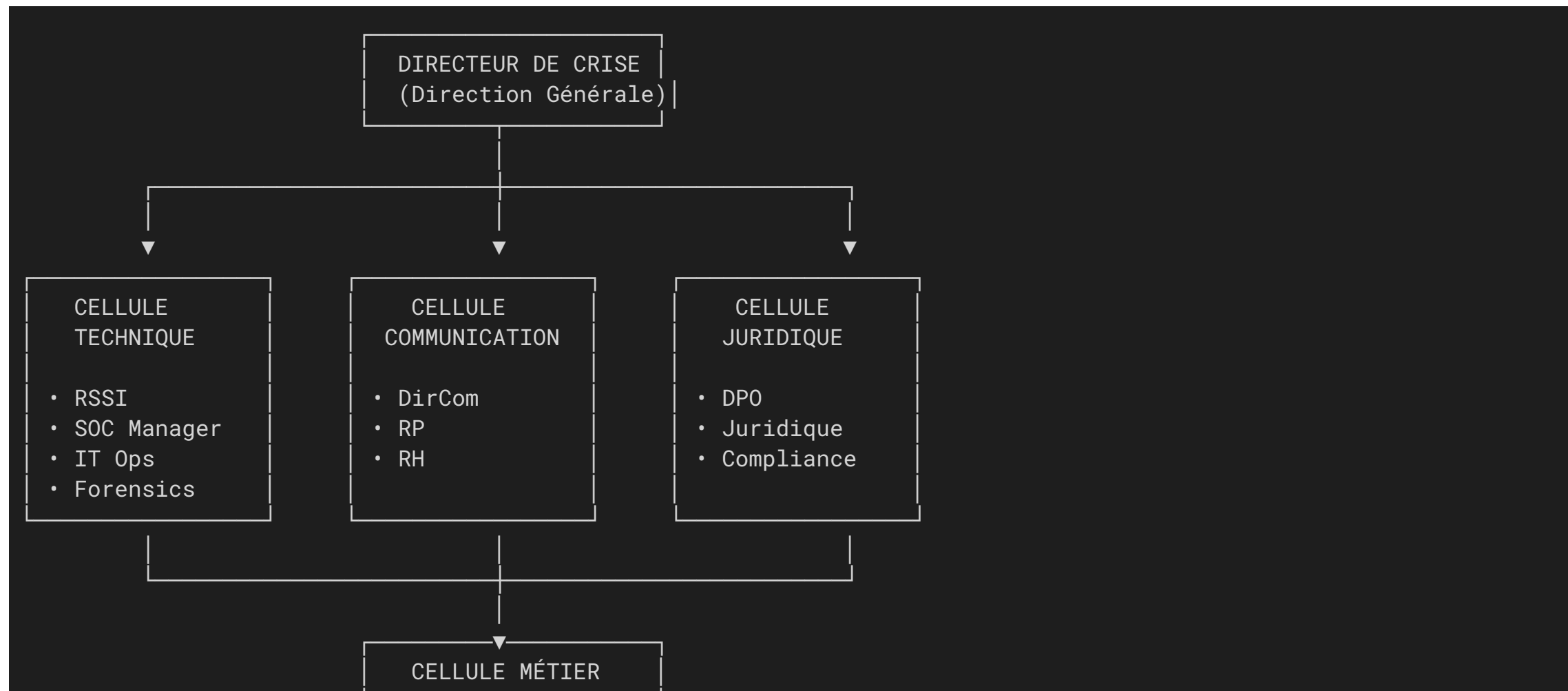
Définition

“ Une **crise cyber** est un incident de sécurité dont l'ampleur, l'impact ou la complexité dépasse les capacités de réponse normales de l'organisation. ”

Critères d'activation de la cellule de crise

- Impact sur la continuité d'activité (arrêt production)
- Atteinte à la réputation (médias, réseaux sociaux)
- Exposition de données sensibles massives

Structure de la Cellule de Crise



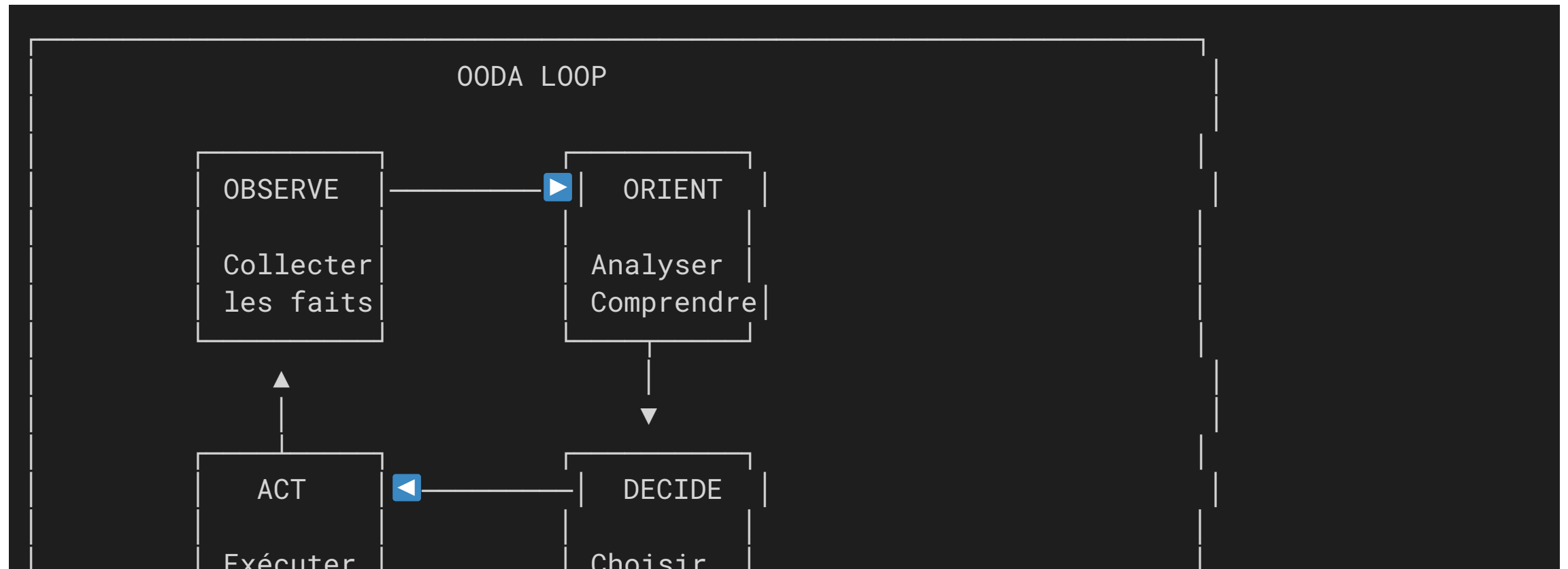
Rôles et Responsabilités (RACI)

Activité	Dir. Crise	RSSI	IT Ops	DPO	DirCom
Décision de communication	A	C	I	C	R
Confinement technique	I	A	R	I	I
Notification CNIL	A	C	I	R	C
Décision de payer rançon	R/A	C	I	C	I
Restauration systèmes	I	A	R	I	I
Communication clients	A	I	I	C	R
Investigation forensique	I	A	C	C	I

“ **R** = Responsable, **A** = Accountable, **C** = Consulted, **I** = Informed ”

Prise de Décision Sous Pression

Framework OODA Loop



Gestion des Priorités en Crise

Matrice d'Eisenhower adaptée

URGENT + IMPORTANT

Faire immédiatement

- Confinement systèmes critiques
- Protection des données clients
- Communication autorités (si requis)

URGENT + NON IMPORTANT

Déléguer

- Répondre aux demandes internes
- Mises à jour de statut
- Coordination logistique

NON URGENT + NON

NON URGENT + IMPORTANT

NON URGENT + NON IMPORTANT

Démo : Dashboard de Gestion de Crise

```
#!/usr/bin/env python3
"""
Dashboard de gestion de crise en temps réel
Centralise les informations critiques pour la cellule de crise
"""

from dataclasses import dataclass
from datetime import datetime
from typing import List, Dict
from enum import Enum

class CrisisStatus(Enum):
    DETECTION = "detection"
    TRIAGE = "triage"
    CONTAINMENT = "containment"
    ERADICATION = "eradication"
    RECOVERY = "recovery"
    CLOSED = "closed"

@dataclass
class CrisisDashboard:
    incident_id: str
    status: CrisisStatus
    start_time: datetime
    severity: str
    affected_systems: List[str]
    affected_users: int
    business_impact: str
    current_actions: List[str]
    decisions_pending: List[str]
    timeline: List[Dict]

    def get_duration(self) -> str:
        """Retourne la durée de la crise"""
        delta = datetime.now() - self.start_time
        hours = delta.seconds // 3600
        minutes = (delta.seconds % 3600) // 60
        return f"{delta.days}j {hours}h {minutes}m"

    def add_timeline_entry(self, action: str, actor: str):
        """Ajoute une entrée à la timeline"""
        self.timeline.append({
            "timestamp": datetime.now().isoformat(),
            "action": action,
            "actor": actor
        })
```

Démo : Dashboard (suite)

```
def generate_sitrep(self) -> str:
    """Génère un rapport de situation (SITREP)"""

    sitrep = f"""

    SITUATION REPORT (SITREP)

    Incident ID: {self.incident_id:<20} Durée: {self.get_duration():<15}
    Statut: {self.status.value:<25} Sévérité: {self.severity:<10}

    IMPACT
    • Systèmes affectés: {len(self.affected_systems):<10}
    • Utilisateurs impactés: {self.affected_users:<10}
    • Impact business: {self.business_impact:<30}

    ACTIONS EN COURS

    """

    for action in self.current_actions[:5]:
        sitrep += f"    • {action:<60}\n"

    sitrep += "    "

    DÉCISIONS EN ATTENTE

    """

    for decision in self.decisions_pending[:3]:
        sitrep += f"    ⚠ {decision:<60}\n"

    sitrep += "    "
```

Communication en Temps de Crise

Les 5 Règles d'Or

1. Rapidité

Communiquer vite, même si l'information est partielle

2. Transparence

Ne pas minimiser, ne pas mentir

3. Cohérence

Un seul porte-parole, messages alignés

4. Empathie

Reconnaître l'impact sur les parties prenantes

5. Proactivité

Anticiper les questions, préparer les Q&A

Attention : En France, notification CNIL obligatoire sous 72h si

Templates de Communication

Communication Interne (Employés)

OBJET: [URGENT] Incident de sécurité en cours - Instructions

Chers collègues,

Notre équipe de sécurité a détecté un incident de sécurité affectant [description générale]. Nous avons immédiatement activé notre plan de réponse aux incidents.

ACTIONS IMMÉDIATES REQUISES:

- Ne pas ouvrir de pièces jointes suspectes
- Signaler toute anomalie à security@company.com
- [Autres instructions spécifiques]

CE QUE NOUS FAISONS:

- Notre équipe travaille 24/7 pour résoudre la situation
- Des mises à jour régulières seront communiquées

Templates de Communication (suite)

Communication Externe (Clients)

NOTIFICATION IMPORTANTE - Incident de Sécurité

Cher(e) client(e),

Nous vous informons qu'un incident de sécurité a été détecté le [date].
Par souci de transparence, nous souhaitons vous communiquer les informations suivantes:

CE QUI S'EST PASSÉ:

[Description factuelle et mesurée]

DONNÉES POTENTIELLEMENT CONCERNÉES:

[Liste précise des types de données]

CE QUE NOUS AVONS FAIT:

- Activation immédiate de notre plan de réponse
- Engagement d'experts en cybersécurité
- Notification aux autorités compétentes
- [Autres mesures]

CE QUE VOUS POUVEZ FAIRE:

- Changer votre mot de passe par précaution

Exercice 4 : Tabletop Exercise

Scénario

Jour J, 14h00 : Votre SOC détecte une activité anormale sur le serveur de base de données clients.

14h15 : Confirmation d'une exfiltration de données en cours. 2 millions de records clients (nom, email, historique d'achats).

14h30 : Un journaliste contacte votre service presse : "Nous avons reçu des informations selon lesquelles vos données clients sont en vente sur le dark web."

14h45 : L'attaquant envoie un email de demande de rançon.

Déroulement (2h)

Module 5

Activités Post-Incident

Importance du Post-Incident

Pourquoi c'est critique ?

Sans post-mortem

- Même attaque peut se reproduire
- Pas d'amélioration des défenses
- Perte de connaissance institutionnelle
- Frustration des équipes

Avec post-mortem

- Renforcement des contrôles
- Capitalisation de l'expérience
- Amélioration continue
- Culture de sécurité renforcée

Structure d'un Rapport Post-Incident

RAPPORT POST-INCIDENT

1. RÉSUMÉ EXÉCUTIF
 - └ Impact, durée, coût estimé
2. CHRONOLOGIE DÉTAILLÉE
 - └ Timeline minute par minute
3. ANALYSE TECHNIQUE
 - └ Vecteur d'attaque, TTPs, IOCs
4. ROOT CAUSE ANALYSIS
 - └ Causes profondes (5 Whys, Ishikawa)
5. IMPACT ASSESSMENT
 - └ Business, données, réputation
6. LESSONS LEARNED
 - └ Ce qui a bien/mal fonctionné
7. RECOMMANDATIONS
 - └ Actions correctives avec responsables et délais

Méthode des 5 Pourquoi (5 Whys)

Exemple Pratique

PROBLÈME: Un ransomware a chiffré les serveurs de production

POURQUOI 1: Parce qu'un employé a ouvert une pièce jointe malveillante

↓

POURQUOI 2: Parce que l'email a passé les filtres anti-spam

↓

POURQUOI 3: Parce que les règles de filtrage n'étaient pas à jour

↓

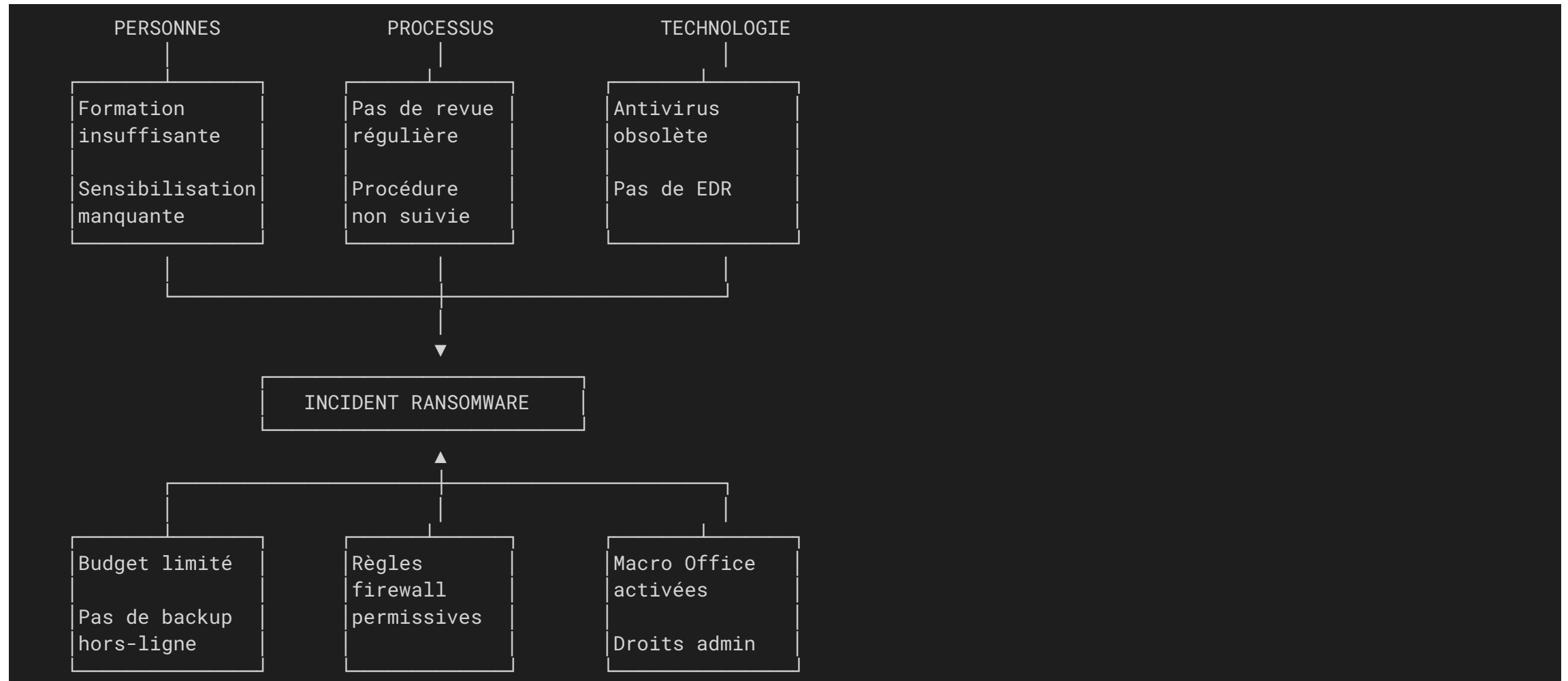
POURQUOI 4: Parce qu'il n'y a pas de processus de mise à jour régulier

↓

POURQUOI 5: Parce que la responsabilité n'est pas clairement assignée

CAUSE RACINE: Absence de gouvernance sur la maintenance des contrôles

Diagramme d'Ishikawa (Causes-Effets)



Démo : Générateur de Rapport Post-Incident

```
#!/usr/bin/env python3
"""
Générateur automatique de rapport post-incident
"""

from dataclasses import dataclass
from datetime import datetime
from typing import List, Dict
import json

@dataclass
class IncidentReport:
    incident_id: str
    title: str
    severity: str
    start_time: datetime
    end_time: datetime
    affected_systems: List[str]
    attack_vector: str
    ttps: List[str] # MITRE ATT&CK
    iocs: List[Dict]
    timeline: List[Dict]
    root_causes: List[str]
    lessons_learned: Dict[str, List[str]] # what_worked, what_didnt
    recommendations: List[Dict]

    def calculate_mttr(self) -> float:
        """Mean Time To Recover (en heures)"""
        delta = self.end_time - self.start_time
        return delta.total_seconds() / 3600

    def generate_executive_summary(self) -> str:
        """Génère le résumé exécutif"""

        mttr = self.calculate_mttr()

        summary = f"""
## Résumé Exécutif

**Incident**: {self.title}
**ID**: {self.incident_id}
**Sévérité**: {self.severity}

```

Démo : Rapport Post-Incident (suite)

```
def generate_full_report(self) -> str:
    """Génère le rapport complet en Markdown"""

    report = f"""

    # Rapport Post-Incident: {self.incident_id}

    Date de génération: {datetime.now().strftime('%Y-%m-%d %H:%M')}

    ---

    {self.generate_executive_summary()}

    ---

    ## Chronologie Détaillée

    | Heure | Événement | Acteur |
    |-----|-----|-----|
    """

    for entry in self.timeline:
        report += f"| {entry['time']} | {entry['event']} | {entry['actor']} |\n"

    report += f"""

    ---

    ## Analyse Technique

    ### Vecteur d'Attaque
    {self.attack_vector}

    ### TTPs Identifiés (MITRE ATT&CK)
    """

    for ttp in self.ttps:
        report += f"- {ttp}\n"

    report += """

    ### Indicateurs de Compromission (IOCs)
    """

    for ioc in self.iocs:
```

Démo : Rapport - Recommandations

```
def generate_recommendations_table(self) -> str:
    """Génère le tableau des recommandations"""

    table = ""
    ## Recommendations

    | # | Recommandation | Priorité | Responsable | Échéance | Statut |
    |---|-----|-----|-----|-----|-----|
    """

    for i, rec in enumerate(self.recommendations, 1):
        table += f"| {i} | {rec['action']} | {rec['priority']} | {rec['owner']} | {rec['deadline']} | {rec['status']} |\n"

    return table

# Exemple d'utilisation
report = IncidentReport(
    incident_id="INC-2024-042",
    title="Ransomware LockBit sur serveurs de fichiers",
    severity="P1 - Critique",
    start_time=datetime(2024, 1, 15, 9, 15),
    end_time=datetime(2024, 1, 16, 18, 30),
    affected_systems=["FILE-SRV-01", "FILE-SRV-02", "BACKUP-SRV-01"],
    attack_vector="Phishing email avec macro malveillante",
    ttps=["T1566.001 - Spearphishing Attachment",
          "T1059.001 - PowerShell",
          "T1486 - Data Encrypted for Impact"],
    iocs=[
        {"type": "hash_sha256", "value": "a1b2c3d4e5f6..."},
        {"type": "domain", "value": "malicious-c2.evil.com"},
        {"type": "ip", "value": "185.234.72.100"}
    ],
    timeline=[...],
    root_causes=["Manque de sensibilisation", "Macros activées par défaut"],
    lessons_learned={...},
    recommendations=[...]
```

Métriques de Performance (KPIs)

KPIs Essentiels pour un CSIRT

Métrique	Description	Cible
MTTD	Mean Time To Detect	< 24h
MTTR	Mean Time To Respond	< 4h (P1)
MTTC	Mean Time To Contain	< 1h (P1)
MTTRE	Mean Time To Recover	< 48h
Taux de faux positifs	Alertes non confirmées	< 30%
Taux de récurrence	Incidents similaires répétés	< 5%
Couverture	Assets monitorés / Total	> 95%

Démo : Dashboard de Métriques

```
#!/usr/bin/env python3
"""
Calcul automatique des métriques CSIRT
"""

from datetime import datetime, timedelta
from typing import List
import statistics

class CSIRTMetrics:
    def __init__(self, incidents: List[dict]):
        self.incidents = incidents

    def calculate_mtttd(self) -> float:
        """Mean Time To Detect (heures)"""
        detection_times = []
        for inc in self.incidents:
            if inc.get('compromise_time') and inc.get('detection_time'):
                delta = inc['detection_time'] - inc['compromise_time']
                detection_times.append(delta.total_seconds() / 3600)
        return statistics.mean(detection_times) if detection_times else 0

    def calculate_mttr(self) -> float:
        """Mean Time To Respond (heures)"""
        response_times = []
        for inc in self.incidents:
            if inc.get('detection_time') and inc.get('response_time'):
                delta = inc['response_time'] - inc['detection_time']
                response_times.append(delta.total_seconds() / 3600)
        return statistics.mean(response_times) if response_times else 0

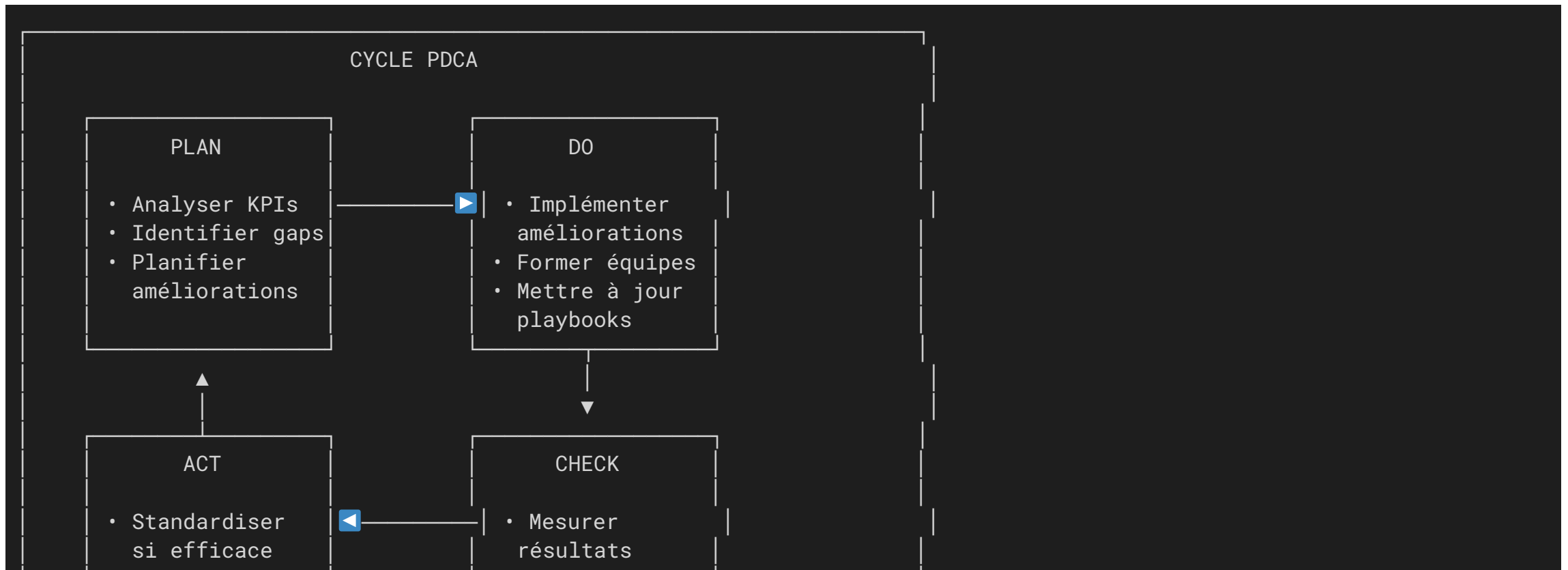
    def calculate_false_positive_rate(self) -> float:
        """Taux de faux positifs"""
        total = len(self.incidents)
        fps = sum(1 for inc in self.incidents if inc.get('false_positive'))
        return (fps / total * 100) if total > 0 else 0

    def generate_monthly_report(self) -> str:
        """Génère le rapport mensuel de métriques"""
        return f"""
MÉTRIQUES CSIRT - {datetime.now().strftime('%B %Y')}
=====

MTTD (Mean Time To Detect):    {self.calculate_mtttd():.1f}h
MTTR (Mean Time To Respond):   {self.calculate_mttr():.1f}h
Taux de faux positifs:        {self.calculate_false_positive_rate():.1f}%
        """
```

Amélioration Continue

Cycle PDCA appliqué à la Réponse aux Incidents



Exercice 5 : Rédaction Post-Mortem

Consigne

À partir du scénario de l'exercice 4 (exfiltration de données + ransomware), rédigez un rapport post-mortem complet.

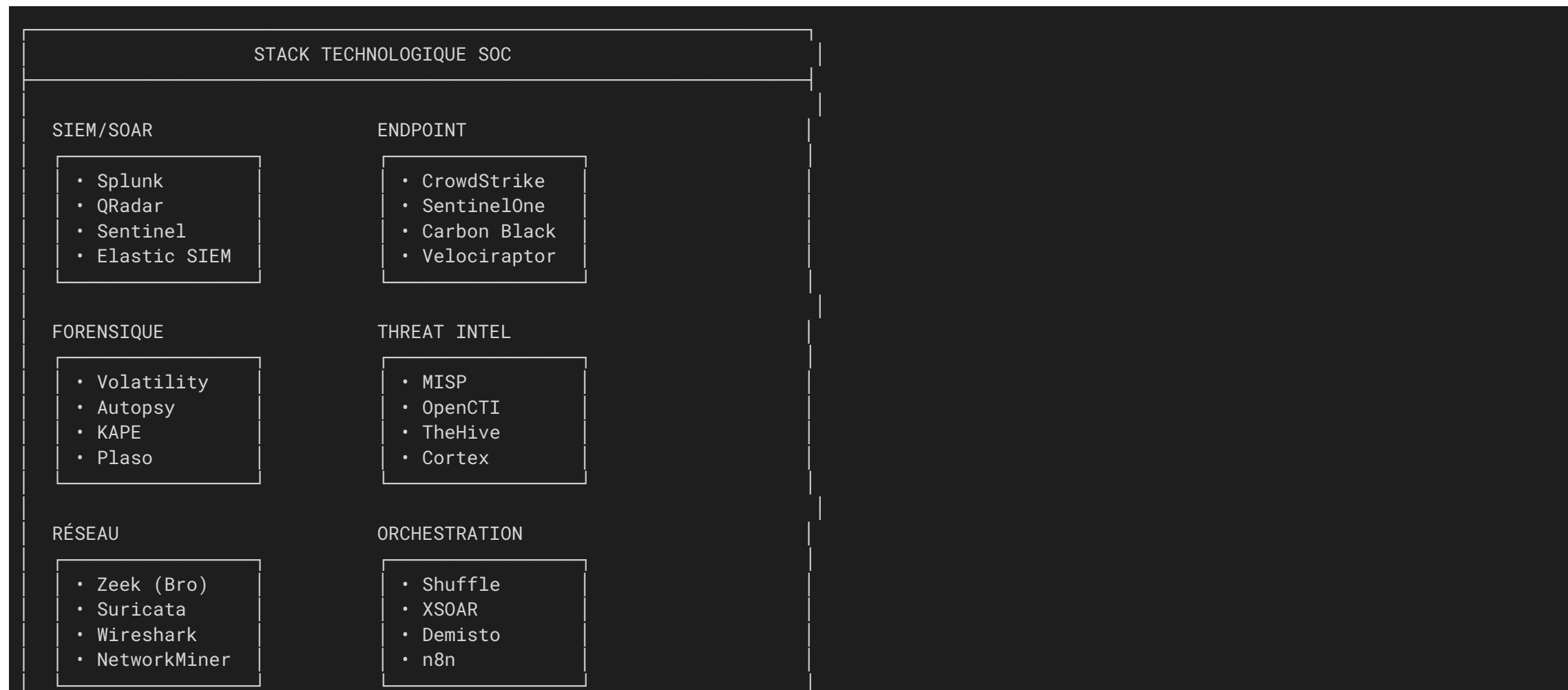
Sections à compléter

1. **Résumé exécutif** (1/2 page)
2. **Chronologie** (timeline détaillée)
3. **Analyse des causes** (5 Whys + Ishikawa)
4. **Lessons learned** (ce qui a fonctionné / ce qui n'a pas fonctionné)

Module 6

Outils et Exercices Pratiques

Écosystème d'Outils SOC/CSIRT



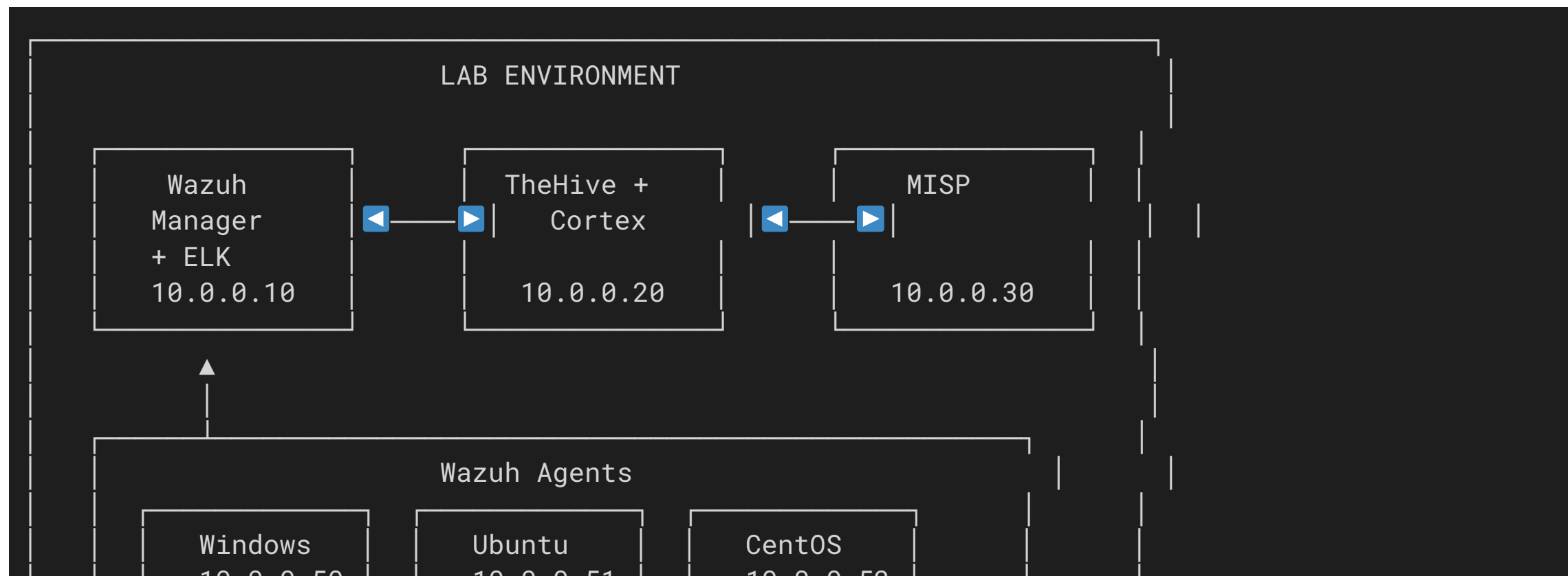
Outils Open Source Recommandés

Stack complète gratuite

Catégorie	Outil	Description
SIEM	Wazuh + ELK	Détection, corrélation, dashboards
SOAR	Shuffle	Automatisation des playbooks
Threat Intel	MISP	Partage d'IOC, corrélation
Case Management	TheHive	Gestion des incidents
Forensique	Volatility 3	Analyse mémoire
Forensique	KAPE	Collecte de preuves Windows
Network	Zeek	Analyse de trafic réseau
EDR	Velociraptor	Investigation endpoint
Sandbox	Cuckoo	Analyse dynamique de malware

Lab 1 : Installation Stack SOC

Architecture du Lab



Lab 1 : Script d'Installation Wazuh

```
#!/bin/bash
# Installation Wazuh All-in-One avec Docker
# Prérequis: Docker + Docker Compose

# Cloner le repository
git clone https://github.com/wazuh/wazuh-docker.git -b v4.7.0

cd wazuh-docker/single-node

# Générer les certificats
docker-compose -f generate-indexer-certs.yml run --rm generator

# Démarrer la stack
docker-compose up -d

# Vérifier le statut
docker-compose ps

echo "====="
echo "Wazuh Dashboard: https://localhost:443"
echo "Username: admin"
echo "Password: SecretPassword"
echo "====="

# Installation de l'agent sur une machine cible
# Sur Ubuntu/Debian:
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --no-default-keyring \
  --keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg --import && chmod 644 \
  /usr/share/keyrings/wazuh.gpg

echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ \
  stable main" | tee /etc/apt/sources.list.d/wazuh.list

apt-get update && apt-get install wazuh-agent

# Configurer l'agent
WAZUH_MANAGER='10.0.0.10' WAZUH_REGISTRATION_SERVER='10.0.0.10' \
```

Lab 2 : Création de Règles de Détection

Règle Wazuh personnalisée

```
<!-- /var/ossec/etc/rules/local_rules.xml -->

<group name="local,syslog,sshd,">

  <!-- Détection brute force SSH améliorée -->
  <rule id="100001" level="10">
    <if_sid>5716</if_sid>
    <same_source_ip />
    <description>ALERTE: Possible attaque brute force SSH (5+ échecs)</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
    <group>authentication_failures,</group>
  </rule>

  <!-- Détection élévation de privilèges suspecte -->
  <rule id="100002" level="12">
    <if_group>syslog</if_group>
    <match>sudo.*COMMAND=</match>
    <regex>\bin\bash|\bin\sh|passwd|shadow|sudoers</regex>
    <description>CRITIQUE: Commande sudo suspecte détectée</description>
    <mitre>
      <id>T1548.003</id>
    </mitre>
    <group>privilege_escalation,</group>
  </rule>

  <!-- Détection exfiltration via curl/wget -->
  <rule id="100003" level="10">
    <if_group>syslog</if_group>
```

Lab 2 : Test des Règles

```
#!/bin/bash
# Script de test des règles de détection

echo "[*] Test 1: Simulation brute force SSH"
for i in {1..6}; do
    ssh invalid_user@localhost 2>/dev/null
done

echo "[*] Test 2: Simulation commande sudo suspecte"
sudo cat /etc/shadow 2>/dev/null

echo "[*] Test 3: Simulation exfiltration (bénigne)"
echo "test data" | curl -X POST -d @- https://httpbin.org/post

echo "[*] Vérification des alertes dans Wazuh..."
sleep 5

# Requête API Wazuh pour vérifier les alertes
curl -k -X GET "https://localhost:55000/alerts" \
    -H "Authorization: Bearer $TOKEN" \
    | jq '.data.affected_items | .[] | select(.rule.id >= 100001)'
```

Lab 3 : Playbook SOAR avec Shuffle

Playbook de Réponse Automatique

```
# Playbook: Réponse automatique à une détection de malware
# Platform: Shuffle SOAR

name: "Malware Detection Response"
description: "Playbook automatisé de réponse à une détection de malware"
triggers:
  - type: webhook
    name: wazuh_malware_alert

actions:
  - name: "1. Enrichir IOC"
    app: virustotal
    action: scan_file_hash
    parameters:
      hash: "$trigger.file_hash"

  - name: "2. Créer ticket TheHive"
    app: thehive
    action: create_case
    parameters:
      title: "Malware détecté - $trigger.hostname"
      severity: 3
      tags: ["malware", "automated"]
      description: |
        Malware détecté sur $trigger.hostname
        Hash: $trigger.file_hash
        Path: $trigger.file_path
        VT Score: $action_1.positives / $action_1.total

  - name: "3. Isoler endpoint"
    app: crowdstrike
    action: contain_host
    parameters:
      hostname: "$trigger.hostname"
    condition: "$action_1.positives > 5"
```

Lab 4 : Analyse Forensique avec Velociraptor

Collecte de Preuves Windows

```
# Artifact Velociraptor personnalisé
name: Custom.Windows.Triage.Full
description: |
  Collecte complète pour analyse forensique Windows

parameters:
  - name: OutputPath
    default: C:\Temp\forensics

sources:
  - name: MemoryAcquisition
    query: |
      SELECT * FROM Artifact.Windows.Memory.Acquisition()

  - name: EventLogs
    query: |
      SELECT * FROM Artifact.Windows.EventLogs.EvtxHunter(
        EvtxGlob="C:/Windows/System32/winevt/Logs/*.evtx",
        SearchTerms=["powershell", "cmd.exe", "mimikatz", "psexec"]
      )

  - name: Prefetch
    query: |
      SELECT * FROM Artifact.Windows.Forensics.Prefetch()
```

Lab 5 : Simulation d'Attaque (Purple Team)

Script de Simulation ATT&CK

```
#!/usr/bin/env python3
"""
Simulation d'attaque pour test de détection
ATTENTION: À utiliser uniquement dans un environnement de lab contrôlé
"""

import subprocess
import os
import time

class AttackSimulation:
    """Simule des TTPs MITRE ATT&CK pour tester la détection"""

    def __init__(self):
        self.results = []

    def t1059_001_powershell(self):
        """T1059.001 - Command and Scripting Interpreter: PowerShell"""
        print("[*] Exécution T1059.001 - PowerShell encoded command")

        # Commande bénigne encodée (whoami)
        encoded = "dwBoAG8AYQBtAGkA"
        cmd = f'powershell -EncodedCommand {encoded}'

        try:
            result = subprocess.run(cmd, shell=True, capture_output=True, timeout=10)
            self.results.append(("T1059.001", "SUCCESS"))
        except Exception as e:
            self.results.append(("T1059.001", f"FAILED: {e}"))

    def t1053_005_scheduled_task(self):
        """T1053.005 - Scheduled Task/Job: Scheduled Task"""
        print("[*] Exécution T1053.005 - Création tâche planifiée")

        task_name = "SecurityTest " + str(int(time.time()))
```

Lab 5 : Simulation (suite)

```
def t1003_001_credential_access(self):
    """T1003.001 - OS Credential Dumping: LSASS Memory (SIMULATION ONLY)"""
    print("[*] Exécution T1003.001 - Simulation accès LSASS")

    # SIMULATION BÉNIGNE - Juste ouvrir le processus en lecture
    # Ne fait PAS de dump réel
    cmd = 'tasklist /FI "IMAGENAME eq lsass.exe"'

    try:
        result = subprocess.run(cmd, shell=True, capture_output=True, timeout=10)
        self.results.append(("T1003.001", "SUCCESS (simulated)"))
    except Exception as e:
        self.results.append(("T1003.001", f"FAILED: {e}"))

def run_all(self):
    """Exécute toutes les simulations"""
    print("=" * 50)
    print("DÉMARRAGE SIMULATION ATT&CK")
    print("=" * 50)

    self.t1059_001_powershell()
    time.sleep(2)
    self.t1053_005_scheduled_task()
    time.sleep(2)
    self.t1003_001_credential_access()

    print("\n" + "=" * 50)
    print("RÉSULTATS")
    print("=" * 50)
    for technique, status in self.results:
        print(f" {technique}: {status}")

    print("\n[!] Vérifiez maintenant les alertes dans votre SIEM")
```

Exercice Final : CTF Incident Response

Scénario Complet

Vous êtes l'analyste SOC de garde. À 23h45, plusieurs alertes se déclenchent.

Éléments fournis :

- Dump mémoire d'un serveur compromis
- Logs SIEM des dernières 24h
- Capture réseau (PCAP)
- Image disque du poste patient zéro

Objectifs :

CTF : Étapes Guidées

Étape 1 : Analyse Mémoire

```
# Identifier le profil système
vol -f memory.dmp windows.info

# Lister les processus
vol -f memory.dmp windows.pslist > pslist.txt
vol -f memory.dmp windows.pstree > pstree.txt

# Rechercher des processus suspects
vol -f memory.dmp windows.malfind > malfind.txt

# Extraire les connexions réseau
vol -f memory.dmp windows.netscan > netscan.txt
```

Ressources pour Aller Plus Loin

Formations et Certifications

Certifications

- GIAC GCIH (Incident Handler)
- GIAC GCFA (Forensic Analyst)
- EC-Council ECIH
- CompTIA CySA+
- SANS FOR508

Plateformes de Practice

- Hack The Box (SOC Path)
- Blue Team Labs Online
- CyberDefenders
- LetsDefend
- RangeForce

Documentation

Synthèse

Points Clés à Retenir

Concept	Point Essentiel
Préparation	La qualité de la réponse dépend de la préparation
Détection	Plus vite = moins de dégâts (réduire le MTTD)
Triage	Classifier rapidement pour prioriser
Confinement	Limiter la propagation SANS détruire les preuves
Éradication	S'assurer d'éliminer TOUTE la persistance
Recovery	Valider AVANT de remettre en production
Communication	Transparence et cohérence
Post-Mortem	Blameless + amélioration continue
Métriques	Ce qui ne se mesure pas ne s'améliore pas

Checklist de l'Incident Responder

Préparation

- ☐ Plan de réponse à jour
- ☐ Playbooks par type d'incident
- ☐ Contacts d'escalade disponibles
- ☐ Outils forensiques prêts
- ☐ Exercices réguliers

Détection

- ☐ Alertes correctement configurées
- ☐ Couverture des assets critiques
- ☐ Intégration threat intel
- ☐ Processus de triage défini

Réponse

- ☐ Documentation temps réel
- ☐ Préservation des preuves
- ☐ Communication régulière
- ☐ Décisions tracées

Post-Incident

- ☐ Post-mortem systématique
- ☐ Recommandations suivies
- ☐ Métriques calculées
- ☐ Leçons partagées

Évaluation

Modalité : Projet avec Soutenance

Sujet : Simulation complète de gestion d'incident

Livrables attendus :

1. Plan de réponse aux incidents adapté au contexte
2. Playbooks pour 3 types d'incidents
3. Rapport d'incident complet (scénario fourni)
4. Présentation du post-mortem

Critères d'évaluation :

Questions ?

Merci pour votre attention !

Contact : security@utopios.com

Annexes

A1 : Template Plan de Réponse aux Incidents

Disponible sur le repository du cours :

```
/templates/incident_response_plan_template.md
```

A2 : Playbooks de Référence

- Ransomware Response Playbook
- Phishing Response Playbook
- Data Breach Response Playbook
- DDoS Response Playbook